

Proyecto Especial

Diseño e Implementación de un Lenguaje

C --



Valentín Ruiz - Legajo 64046 - vruiz@itba.edu.ar
Santiago Allegretti - Legajo 65531 - sallegretti@itba.edu.ar

Índice

1. Introducción	1
2. Modelo Computacional	1
2.1. Dominio	1
2.2. Lenguaje	1
3. Implementación	3
3.1. Frontend	3
3.1.1. Análisis Léxico (Flex)	3
3.1.2. Análisis Sintáctico (Bison)	3
3.2. Backend	4
3.2.1. Análisis Semántico ('SymbolTable.c', 'SemanticAnalysis.c')	4
3.2.2. Generación de Código ('Generator.c')	6
3.3. Dificultades Encontradas	8
4. Futuras Extensiones	9
5. Conclusiones	9
6. Referencias	10
7. Bibliografía	10

1. Introducción

El presente trabajo consiste en el diseño e implementación de un DSL denominado **C--**, junto con su compilador. El proyecto se desarrolló en tres etapas: diseño del lenguaje, construcción del frontend (analizador léxico y sintáctico), y construcción del backend (análisis semántico y generación de código).

C-- es un lenguaje orientado a objetos cuyo compilador traduce programas a código C equivalente. Su objetivo es ofrecer abstracciones de programación orientada a objetos (clases, herencia, visibilidad, sobrecarga de métodos) sobre una base de ejecución en C, sin requerir cambios en el entorno de compilación subyacente.

El compilador se construye sobre el proyecto base Flex-Bison-Compiler v2.0.0 (<https://github.com/agustin-golmar/Flex-Bison-Compiler>), utilizando Flex para el análisis léxico y Bison para el análisis sintáctico, con el código fuente en C.

2. Modelo Computacional

2.1. Dominio

El lenguaje se enmarca en el dominio de la programación de sistemas con abstracciones orientadas a objetos. El objetivo es facilitar la construcción de programas estructurados sobre una base de ejecución en C, incorporando mecanismos para modelar entidades mediante clases que agrupan estado y comportamiento.

En C, estas estructuras deben resolverse manualmente mediante `struct` y funciones asociadas, sin garantías de encapsulamiento ni de herencia. C-- abstrae esas limitaciones: los programas se conciben como conjuntos de objetos que interactúan mediante invocaciones de métodos, con estado interno controlado y visibilidad declarada explícitamente.

Los programas escritos en este DSL son traducidos a código C equivalente, permitiendo su compilación y ejecución eficiente.

2.2. Lenguaje

C-- ofrece las siguientes construcciones:

- **Clases:** definición con identificador único; contienen atributos y métodos.
- **Tipos primitivos:** `int`, `bool`, `char`, `float`, `double`, `string`, `void`.
- **Modificadores de tipo:** `signed`, `unsigned`, `short`, `long` (aplicables según reglas C: `signed/unsigned` solo en `int/char`; `short` solo en `int`; `long` solo en `int/double`).
- **Punteros y arrays:** declaración de punteros (τ^*) y arrays de tamaño fijo ($\tau[N]$), con operadores `&` (address-of) y `*` (dereference); acceso por índice (`arr[i]`).
- **Visibilidad:** `public`, `private`, `protected` para atributos, métodos y constructores. La visibilidad es obligatoria en todo miembro de clase; el lenguaje no define una visibilidad por defecto.

- **Herencia simple:** extends; no existe super.
- **Sobrecarga de métodos:** múltiples métodos con el mismo nombre diferenciados por tipo y cantidad de parámetros.
- **Constructores:** inicialización de objetos mediante new.
- **'this':** referencia a la instancia actual dentro de métodos.
- **Métodos y atributos estáticos.**
- **Funciones libres:** funciones declaradas fuera de clases con la palabra clave function.
- **Estructuras de control:** if, if-else, while, for.
- **Expresiones:** aritméticas (+, -, *, /, %), relacionales (<, >, <=, >=, ==, !=), lógicas (&&, ||, !), asignación (=, +=, -=, *=, /=, %=), incremento/decremento (++, --).

Bloque principal: main { ... }.

Ejemplo de programa:

```
class Animal {
    private string nombre;
    protected int edad;

    public Animal(string n, int e) {
        this.nombre = n;
        this.edad = e;
    }

    public string getNombre() {
        return this.nombre;
    }

    public void setNombre(string n) {
        this.nombre = n;
    }

    protected void crecer() {
        this.edad = this.edad + 1;
    }
}

class Perro extends Animal {
    public string raza;

    public Perro(string n, int e, string r) {
        this.setNombre(n);
        this.edad = e;
        this.raza = r;
    }

    public void cumplirAnios() {
        this.crecer();
    }
}

main {
    Perro p = new Perro("Rocco", 3, "Yorkshire Terrier");
    p.cumplirAnios();
}
```

3. Implementación

3.1. Frontend

El frontend transforma el texto fuente en un AST mediante dos fases.

3.1.1. Análisis Léxico (Flex)

El analizador léxico se implementa en `FlexPatterns.l` y `FlexActions.c`. Opera en modo push: cada regla Flex llama a `pushToken()`, que invoca `yypush_parse()` para alimentar el parser de Bison token a token. El módulo `Frontend.c` ejecuta un loop que llama a `executeLexicalAnalysis()` hasta que el estado de compilación deja de ser `IN_PROGRESS`.

Los tokens reconocidos se agrupan en:

- **Palabras clave:** `class`, `extends`, `new`, `this`, `if`, `else`, `while`, `for`, `return`, `main`, `function`, `public`, `private`, `protected`, `static`, tipos primitivos (`int`, `bool`, `char`, `float`, `double`, `string`, `void`), modificadores (`signed`, `unsigned`, `short`, `long`).
- **Identificadores:** secuencias `[a-zA-Z_][a-zA-Z0-9_]*` que no son palabras clave.
- **Literales:** enteros, flotantes, caracteres, strings, booleanos (`true/false`).
- **Operadores y puntuación:** operadores aritméticos, relacionales, lógicos, de asignación, puntero (`*`, `&`), acceso (`.`, `->`) y corchetes.
- **Ignorados:** espacios, tabulaciones, saltos de línea, comentarios de línea (`//`) y de bloque (`/* ... */`). Los comentarios de bloque usan un contexto Flex `%x` (`MULTILINE_COMMENT`) que consume el contenido hasta el cierre `*/`, permitiendo que se extiendan sobre múltiples líneas. Los strings también se procesan con un contexto dedicado (`STRING_LITERAL`). Éste último acumula el contenido en un buffer dinámico y traduce las secuencias de escape `\n`, `\t`, `\r`, `\\`, `\"`, `\0` y `\'`. Cualquier otra secuencia `\x` se interpreta como el carácter literal correspondiente.
- **Desconocidos:** cualquier lexema no reconocido produce el token `UNKNOWN`, que el parser rechaza.

3.1.2. Análisis Sintáctico (Bison)

La gramática se define en `BisonGrammar.y` y las acciones semánticas en `BisonActions.c`. Es una gramática LALR(1) con dos conflictos `reduce/reduce` conocidos.

Conflictos `reduce/reduce`: surgen de la ambigüedad entre declaración de variable de tipo clase y `expression statement` cuando un `statement` comienza con `IDENTIFIER`. La gramática presenta dos conflictos `reduce/reduce`, ambos sobre el mismo punto de decisión (reducir `IDENTIFIER` a nombre de tipo o a expresión), pero disparados sobre dos token de lookahead distintos: `*` y `[`. Por ejemplo, `x *` y `x [` podría ser una declaración de un puntero (`x` actúa como tipo clase) o una multiplicación. Otro ejemplo podría ser el de `x[5]`, que podría tratarse de una declaración de array (`x[5] arr`, con `x` actuando como tipo clase) o un acceso por índice. Bison resuelve ambos conflictos a favor de la declaración (la primera regla), y el backend

desambigua semánticamente (ver sección 3.3).

Las **acciones semánticas** (`BisonActions.c`) construyen el AST mediante `calloc`. Los nodos principales son:

- **'Program'**: raíz; contiene listas de `Class`, `Function`, `StatementList` del `main`.
- **'Class'**: nombre, padre (`extends`), lista de `Field`, lista de `Method`, `next`.
- **'Method'**: nombre, tipo de retorno, lista de `Parameter`, lista de `Statement` de su cuerpo, visibilidad, flags `static/constructor`, `next`.
- **'Function'**: nombre, tipo de retorno, lista de `Parameter`, lista de `Statement` de su cuerpo, `next`.
- **'Field'**: nombre, tipo, visibilidad, inicializador opcional, `next`.
- **'Parameter'**: nombre, tipo, `next`.
- **'Statement'**: variable declaration, block, if, while, for, return, expression statement.
- **'Expression'**: binary, unary, literal, identifier, field access (`.`), arrow access (`->`), index (`[]`), call, new.
- **'Type'**: kind (enum `TypeKind`), más `inner/arraySize` para punteros/arrays, `className` para clases, flags `isSigned/isUnsigned/isShort/isLong` para modificadores.

La memoria del AST es gestionada por destructores (`destroyProgram`, `destroyExpression`, etc.) invocados desde `EntryPoint.c` al finalizar la compilación.

3.2. Backend

El backend opera sobre el AST producido por el frontend y se divide en dos fases: análisis semántico y generación de código.

3.2.1. Análisis Semántico ('SymbolTable.c', 'SemanticAnalysis.c')

La fase semántica implementa una tabla de símbolos con stack de scopes (`SymbolTable`), que se construye durante el recorrido del AST.

Recolección de declaraciones (`_collectDeclarations`): primera pasada que registra todas las clases (con sus campos y métodos) y funciones libres, antes de validar los cuerpos. Detecta en esta pasada: clases duplicadas, miembros duplicados (campos y firmas de métodos), padres inexistentes, ciclos de herencia, y tipos de campo con modifiers inválidos.

Validación de cuerpos (`_processBodies`): segunda pasada que recorre `main`, funciones y métodos. Por cada cuerpo se abre un scope (`pushScope/popScope`), se declaran los parámetros y se visitan los statements con `_checkStatement`.

'_checkStatement':

- Declaraciones de variable: compatibilidad de tipos (`initializer` vs tipo declarado), array sin tamaño en posición local, modifiers inválidos, `rewrite decl-vs-expr` (ver sección 3.3), shadowing.
- Expresiones: type-check completo vía `resolveExpressionType`.

- `if/while/for`: condición válida.
- `return`: tipo compatible con el retorno esperado del contexto (`currentReturnType`).

'resolveExpressionType': resuelve el tipo de cualquier expresión del AST.

- Literales → tipo primitivo correspondiente.
- Identificadores → lookup en el scope stack.
- Binarios → reglas de coerción numérica (`_numericRank/_widenNumeric`), asignación compatible, operadores relacionales/lógicos.
- Unarios → `!` requiere `bool`, `-/+/--` requieren numérico, `*` requiere puntero/array, `&` requiere `lvalue`.
- Field access / arrow access → lookup de campo respetando visibilidad y herencia; anota `resolvedFieldDepth` para el codegen.
- Call → lookup de método por nombre, aridad y tipos (overload); anota `resolvedMethod` y `resolvedOwnerClass` para el codegen.
- `new` → verifica constructor existente y compatibilidad de argumentos.
- Index (`arr[i]`) → `arr` debe ser puntero/array; `i` debe ser numérico; devuelve el inner type.

Reglas de coerción. Los tipos numéricos se ordenan por rango `char < int < float < double`. En operaciones aritméticas el resultado toma el rango mayor de los operandos (`int + float → float`). En asignaciones solo se permite *widening* (`int → float` es válido; `float → int` se rechaza). El tipo `bool` no participa de coerciones numéricas: solo es compatible con `bool`. Entre clases, una instancia es asignable a su misma clase o a una clase ancestro (upcast). Punteros y arrays son compatibles si coinciden sus tipos apuntados.

Origen → Destino	¿Permitido?
<code>char → int → float → double</code>	Sí (widening)
<code>float → int, double → int</code>	No (narrowing)
<code>bool ↔ int</code>	No
Subclase → Superclase (upcast)	Sí
Superclase → Subclase (downcast)	No

Rewrite decl-vs-expr: el caso `x * y`; (donde `x` no está registrado como clase) se reescribe en el AST como una multiplicación. Se desarrolla en la sección 3.3.

Gestión de memoria de tipos dinámicos: el operador `&x` construye un `Type POINTER` en heap, registrado en `allocatedTypes` y liberado en `destroySymbolTable`. Se desarrolla en la sección 3.3.

Sobrecarga: `_typesEqual` recursa sobre `inner` para `POINTER` y `ARRAY`, garantizando que `f(int*)` y `f(char*)` se traten como firmas distintas. El mangling usa `ptr/arr` como sufijo para `POINTER/ARRAY` respectivamente.

Validación de modificadores de tipo (`validateTypeModifiers`): valida que los flags `isSigned/isUnsigned/isShort/isLong` sean legales para el `kind` base. La función se llama en cada sitio de declaración (campos, variables locales, parámetros, tipos de retorno).

Diagnóstico de errores. Los errores semánticos se reportan mediante `logError` (a `stderr`), con mensajes descriptivos que nombran la entidad involucrada —por ejemplo `Type mismatch in declaration of 'x'. o Class 'Perro' extends undefined class 'Foo'`. Los mensajes no incluyen número de línea, ya que los nodos del AST no conservan la posición del fuente. Ante cualquier error la compilación finaliza con estado distinto de cero, sin generar código.

3.2.2. Generación de Código ('Generator.c')

El generador recorre el AST y emite un programa C válido (`-std=c99`) a `stdout`.

Estructura del archivo C generado:

1. **Prólogo:** includes de `stdlib` (`<stdlib.h>`, `<string.h>`, `<stdio.h>`), definición del registry de memoria (`_xmalloc`, `_freeAll`); su funcionamiento se desarrolla en §3.3.
2. **Forward declarations of structs:** un `typedef struct X X`; por clase.
3. **Prototipos de funciones:** forward-declarations de todos los métodos, constructores y funciones libres, necesarios para evitar errores de uso antes de definición.
4. **Definiciones de structs:** `struct X { ParentClass parent; fields... };`, emitidas en orden topológico de herencia (cada padre antes que sus hijas); el algoritmo se desarrolla en la sección 3.3.
5. **Definiciones de métodos y constructores:** en el formato
`RetType ClassName__methodName__mangling(ClassName * self, params).`
6. **Definiciones de funciones libres.**
7. **'main':** bloque principal, con `atexit(_freeAll)` al inicio.

Decisiones de generación relevantes:

- **Clases como punteros:** las variables de tipo clase se generan como punteros al struct (`ClassName *`). La instanciación (`new X()`) llama al constructor `X__new__mangling(...)` que devuelve `X *`.
- **Herencia por embedding:** la clase hija incluye un campo `ParentClassType parent`; como primer miembro del struct. El acceso a campos heredados se resuelve emitiendo `->parent`. tantas veces como niveles de herencia haya (profundidad anotada en `resolvedFieldDepth`).
- **Mangling de nombres:** para soportar sobrecarga, los métodos se mangnan como `ClassName__methodName__param1type_param2type` (doble guión bajo separa el nombre del método de los parámetros; guión bajo simple separa cada parámetro). En el mangling, los tipos primitivos se reducen a su nombre C subyacente (`int→int`,

bool→bool, char→char, string→charptr, float→float, double→double); POINTER → <inner>ptr; ARRAY → <inner>arr. Los modificadores no participan del mangling.

- **Dispatch estático:** el método a invocar se resuelve en tiempo de compilación a partir del tipo declarado de la variable (resolvedOwnerClass). No hay dispatch virtual (ver secciones 3.3 y 4).
- **Arrays:** la sintaxis de C-- es prefija (int[5] x); la generación la convierte a postfija C (int x[5]). Arrays sin tamaño en parámetros decaen a puntero (int *x).
- **Modificadores de tipo:** _generateTypeName antepone el prefijo en orden canónico C: [signed|unsigned] [short|long] base.
- **Gestión de memoria (runtime):** los objetos se alocan con _xmalloc y se liberan con _freeAll vía atexit. Se desarrolla en la sección 3.3.

Ejemplo de código generado. El programa de §2.2 (Animal/Perro) produce el siguiente C (recortado). Se observan las tres decisiones centrales en acción: el registry de memoria, la herencia por embedding y el mangling de nombres.

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>

/* Registry de memoria: cada objeto se rastrea y se libera al salir. */
static void ** _allocations = NULL;
static int _allocCount = 0;

static void _freeAll(void) {
    for (int i = 0; i < _allocCount; i++) free(_allocations[i]);
    free(_allocations);
}

static void * _xmalloc(size_t size) {
    void * ptr = malloc(size);
    if (!ptr) { fprintf(stderr, "out of memory\n"); exit(1); }
    /* ... registra ptr en _allocations ... */
    _allocations[_allocCount++] = ptr;
    return ptr;
}

/* Forward declarations y prototipos */
typedef struct Perro Perro;
typedef struct Animal Animal;
Perro * Perro__new__charptr_int_charptr(char * n, int e, char * r);
void Animal__setNombre__charptr(Animal * self, char * n);
/* ... resto de prototipos ... */

/* Structs en orden topológico: el padre antes que la hija */
struct Animal { char * nombre; int edad; };
struct Perro { Animal parent; char * raza; }; /* herencia por embedding */

Perro * Perro__new__charptr_int_charptr(char * n, int e, char * r) {
    Perro * self = _xmalloc(sizeof(Perro));
    Animal__setNombre__charptr((Animal *) self, n); /* método heredado: upcast */
    (self->parent.edad = e); /* campo heredado: ->parent. */
    (self->raza = r);
    return self;
}
```

```

void Animal__crecer(Animal * self) {
    (self->edad = (self->edad + 1));
}

int main(void) {
    atexit(_freeAll);
    Perro * p = Perro__new__charptr_int_charptr("Rocco", 3, "Yorkshire Terrier");
    Perro__cumplirAnios((Perro *) p);
}

```

Puntos a destacar:

- **Mangling:** `Perro__new__charptr_int_charptr` y `Animal__setNombre__charptr` codifican los tipos de parámetros (`string` $\rightarrow$ `charptr`), permitiendo sobrecarga.
- **Embedding:** `struct Perro` incluye `Animal` parent como primer miembro. Como parent está primero, un `Perro *` castea directamente a `Animal *` (misma dirección): por eso la llamada a un método heredado se emite como `Animal__setNombre__charptr((Animal *) self, n)`.
- **Campo heredado:** `edad` (declarado en `Animal`) se accede como `self->parent.edad`.
- **Registry de memoria:** `main` registra `atexit(_freeAll)`; el objeto se aloca con `_xmalloc` y se libera al terminar el programa.

3.3. Dificultades Encontradas

Ambigüedad declaración vs. expression statement: el analizador sintáctico no puede distinguir con un solo token de lookahead si un statement que comienza con IDENTIFIER es una declaración de variable de tipo clase o un expression statement. Esto genera dos conflictos reduce/reduce en la gramática, resueltos por defecto a favor de la declaración.

Para `x * y`;: el backend detecta que el nombre del "tipo" no está registrado como clase y reescribe el nodo AST como una expresión de multiplicación.

Para `arr[i] = val`;: el parser falla antes de llegar al backend porque después de IDENTIFIER '[' expr ']' espera un identificador (nombre de variable), pero encuentra `=`. La solución provisoria es envolver la asignación entre paréntesis: `(arr[i] = val)`; . Al ver (al inicio del statement, el parser sabe con certeza que es una expresión y no una declaración. La solución definitiva sería el typedef hack (el lexer devuelve un token distinto para nombres de clase registrados), pero requiere modificar el frontend, por lo que quedó como extensión futura.

Dispatch estático: la resolución de métodos se realiza en tiempo de compilación sobre el tipo declarado de la variable (`resolvedOwnerClass`). Esto implica que en escenarios de herencia como `Animal a = new Perro(...)`, una llamada `a.hablar()` invoca el método de `Animal` y no el de `Perro`. Esta limitación es propia de no implementar un mecanismo de dispatch virtual, lo que habría implicado un rediseño significativo de la generación de código.

Gestión de memoria: la generación de código a C plantea el problema de cuándo liberar los objetos creados con `new`. C no tiene garbage collector, y rastrear el ciclo de vida de cada objeto de forma estática es complejo en presencia de referencias cruzadas entre clases.

La solución adoptada fue un registry global de alocações: `_xmalloc` registra cada puntero en un array dinámico global, y `_freeAll` (registrada con `atexit`) libera todos al terminar. Esto garantiza 0 leaks para todos los programas de prueba, a costo de no liberar objetos durante la ejecución (aceptable para el tipo de programas que maneja C--).

Tipos dinámicos en el análisis semántico: el operador `&x` produce un nuevo `Type POINTER` durante el type-check, con `inner` apuntando al tipo del operando. Esto requiere memoria dinámica durante la fase semántica, que debe liberarse junto con la tabla de símbolos. Se implementó una lista `allocatedTypes` en `SymbolTable` que acumula estos tipos y los libera en `destroySymbolTable`.

Orden topológico de clases en el codegen: los structs de C deben definirse antes que los structs que los referencian. Como las clases hijas contienen el struct del padre como primer campo (`parent`), deben emitirse en orden de herencia (padre antes que hija). Se implementó `_generateClassesInOrder` que itera en pasadas, emitiendo en cada pasada solo las clases cuyo padre ya fue emitido.

4. Futuras Extensiones

- **Dispatch virtual:** implementar vtables (tabla de punteros a función por clase, inicializada en el constructor) para permitir polimorfismo en tiempo de ejecución. Permitiría que `Animal a = new Perro(...); a.hablar();` invoque el método de `Perro`.
- **Array index assignment sin paréntesis:** resolver la ambigüedad gramatical mediante el typedef hack (el lexer distingue entre nombres de clase y nombres de variable), eliminando la necesidad de escribir `(arr[i] = val);`.
- **Literales de array:** inicialización directa `int[3] v = {1, 2, 3};`.
- **Arrays dinámicos (heap):** `new int[n]` con gestión de memoria, complementando los arrays de tamaño fijo en stack.
- **Interfaces:** declaración de contratos sin implementación, para complementar la herencia simple.
- **'super':** referencia explícita a la clase padre en constructores y métodos, para permitir la inicialización de atributos heredados a través de la cadena de constructores.
- **Generics:** parametrización de clases y funciones por tipo, para permitir estructuras de datos reutilizables.

5. Conclusiones

El proyecto resultó en un compilador funcional para C--, que acepta y rechaza correctamente todos los casos de prueba definidos y genera código C válido. La aceptación/rechazo se valida con `test.sh`, que compara el exit status del compilador contra el resultado esperado (18 casos de aceptación, 34 de rechazo). El código C generado se validó compilándolo con `gcc -std=c99 -Wall -Wextra` (sin warnings) y ejecutándolo. La ausencia de fugas de memoria

está garantizada por construcción: toda alocaación pasa por `_xmalloc`, que registra cada puntero, y `_freeAll`, ejecutada vía `atexit`, los libera al terminar; el generador no emite ninguna otra ruta de alocaación. Esto se confirmó empíricamente con la herramienta `valgrind --leak-check=full` en Linux. Los programas que solo definen clases o funciones, al no tener punto de entrada, compilan como unidad de objeto (`gcc -c`) sin generar ejecutable.

El proceso permitió aplicar en un contexto real los conceptos centrales de la materia: gramáticas libres de contexto, análisis léxico con Flex, análisis sintáctico LALR(1) con Bison, construcción de AST, análisis semántico con tabla de símbolos y scopes, y generación de código.

Las decisiones de diseño más significativas fueron la elección del dispatch estático (simplicidad sobre polimorfismo), la gestión de memoria por registry global (corrección sobre eficiencia), y el rewrite `decl-vs-expr` en semántica en lugar de modificar la gramática (reducir el impacto en el frontend). Estas decisiones resultaron fundamentales dentro de las restricciones del proyecto.

6. Referencias

- Golmar, A. (2024). **Flex-Bison-Compiler v2.0.0**. GitHub.
<https://github.com/agustin-golmar/Flex-Bison-Compiler>

7. Bibliografía

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). **Compilers: Principles, Techniques, and Tools** (2nd ed.). Pearson. (el "Dragon Book") Levine, J. (2009). **Flex & Bison**. O'Reilly Media.
- Kernighan, B. W., & Ritchie, D. M. (1988). **The C Programming Language** (2nd ed.). Prentice Hall.